



A Decade of Rustls



Joe Birr-Pixton 🇬🇧



Daniel McCarney 🇨🇦

RustConf 2026 - September 9th

The Past - beginnings



2nd May 2016
First commit



5th June 2016
Interoperates
with most sites



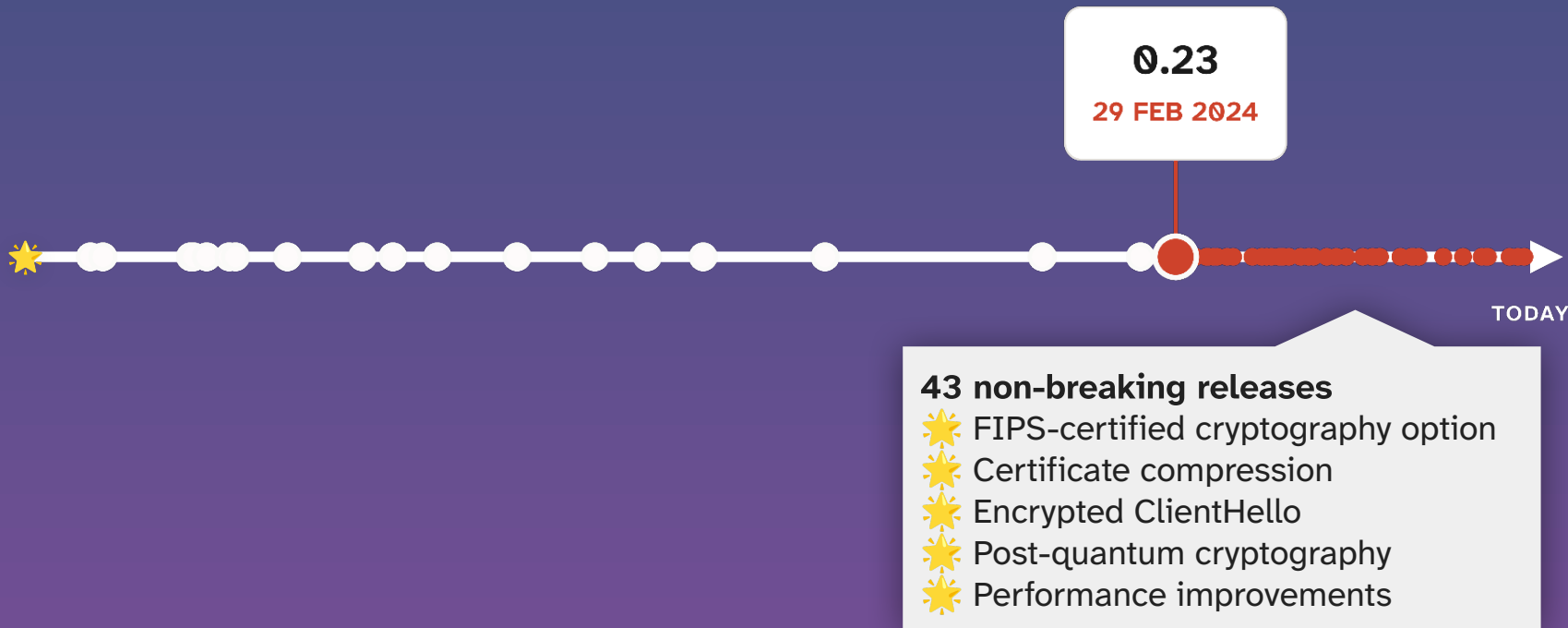
27th August 2016
0.1.0 release



The Past - releases



The Past - releases





The Present

0.24 - the next breaking release

Featuring:

- external buffering
- improvements to bulk data performance
- some support for async-friendly usage
- "split mode"
- new arrangements for choosing a `CryptoProvider`



The Present - external buffering

Forcing everything through `std::io` was not a great idea (copies, wonky errors, no-std, no half-close, poor control of buffer size)

In 0.24:

- all TLS input comes via a `trait TlsInputBuffer`
- all TLS output appended to user-provided `&mut Vec<u8>`
- all incoming plaintext is a borrow into the input buffer (decrypted in-place)

`TlsInputBuffer` impls provided: `SliceInput` and `VecInput`

`VecInput` can read from an `io::Read` like before



The Present - async-friendly support

We've waited for a while for async dyn traits and "async keyword generics"

In 0.24 we'll have a system of session types for steps in the handshake process:

- Servers: NeedsInput → Accepted → VerifyClientIdentity → Complete
- Clients: NeedsInput → VerifyServerIdentity → Complete

Each step can be as async, blocking or completion-based as you like.

More steps can be added in the future (in *technically* non-breaking changes)

(This replaces and builds on the prior Acceptor API from 0.23)



The Present - split mode

Previously we had `Connection` objects that both sent/received data

Now post-handshake sending and receiving are separate objects

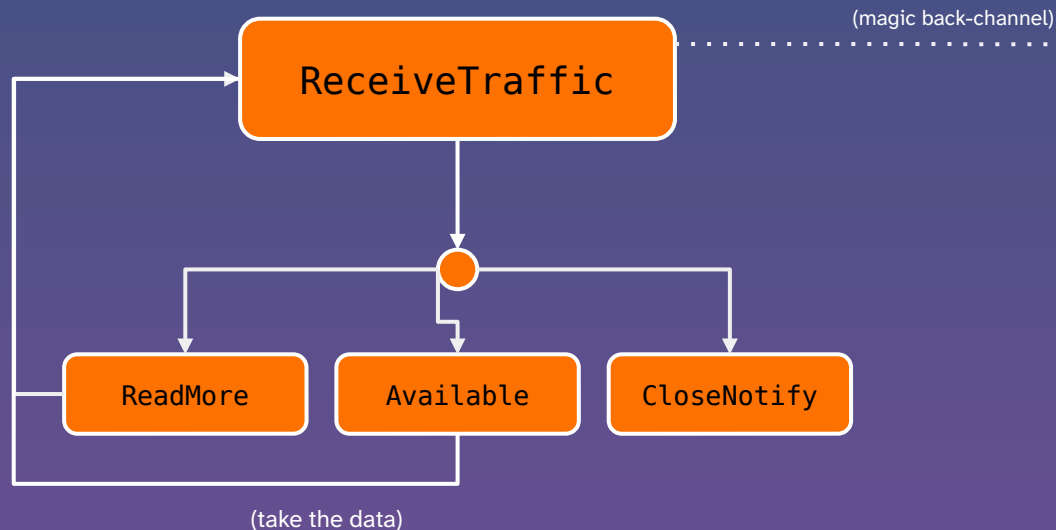
They are both "Send" so can be handled on different threads

Full-duplex workloads can now double throughput

There is a very low contention internal back-channel between them



The Present - split mode

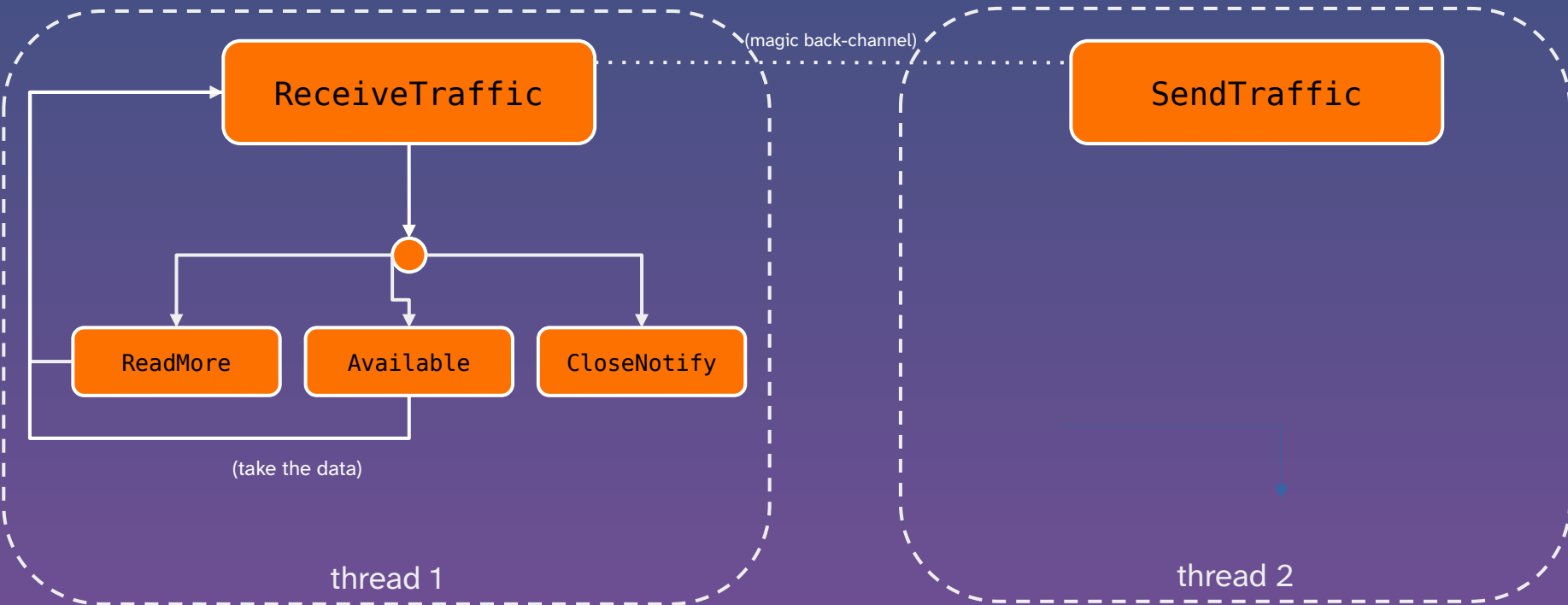


`write()` to encrypt outgoing data
`close()` to stop sending





The Present - split mode





The Present - choosing a CryptoProvider

All cryptography used by Rustls comes via a CryptoProvider

In 0.23 two providers were "built-in" and could be chosen by crate features

In 0.24:

- providers are separate crates (`rustls-aws-lc-rs`, `rustls-ring`, etc.)
- main rustls crate no longer has crate features for selecting a provider
- "choose a global provider at the top of `main()`" needs to happen elsewhere
- (no panics related to feature unification!)



The Future

0.24 will be baked for a little while to shake out any issues

Then 1.0 will follow



0.24



Some ecosystem
baking time



1.0

Thanks!



<https://rustls.dev>

<https://github.com/rustls>



Joe Birr-Pixton 

<https://github.com/ctz>



Daniel McCarney 

<https://github.com/cpu>

Thanks to the Rust Innovation Lab
and Arm for your support!